

## Lab 5: Feature Engineering Techniques with Python

**Objective:** The purpose of this lab was to learn and apply feature engineering techniques to improve model performance using Python. The focus was on handling missing data, encoding categorical variables, addressing class imbalance through oversampling, and evaluating a classification model. These steps are crucial in ensuring the data is well-prepared for machine learning algorithms, ultimately leading to better predictive performance.

### Activities:

1. **Handling Missing Data** One of the main challenges in this lab was dealing with missing values, particularly in the 'deck' column. To address this, missing values were replaced with the category 'Unknown' to prevent any errors during encoding and model training. Before this replacement, we ensured that the column was correctly categorized, which is essential to maintain data consistency. Proper handling of missing values helps prevent the model from making incorrect assumptions due to gaps in the data.

---

Missing values in the dataset:

|             |     |
|-------------|-----|
| survived    | 0   |
| pclass      | 0   |
| sex         | 0   |
| age         | 177 |
| sibsp       | 0   |
| parch       | 0   |
| fare        | 0   |
| embarked    | 2   |
| class       | 0   |
| who         | 0   |
| adult_male  | 0   |
| deck        | 688 |
| embark_town | 2   |
| alive       | 0   |
| alone       | 0   |

dtype: int64

Missing values after imputation:

|             |     |
|-------------|-----|
| survived    | 0   |
| pclass      | 0   |
| sex         | 0   |
| age         | 0   |
| sibsp       | 0   |
| parch       | 0   |
| fare        | 0   |
| embarked    | 0   |
| class       | 0   |
| who         | 0   |
| adult_male  | 0   |
| deck        | 688 |
| embark_town | 0   |
| alive       | 0   |

2. **Encoding Categorical Variables** Since machine learning models work with numerical inputs, it was necessary to convert categorical data into numerical representations. Two encoding techniques were applied based on the type of categorical data:

- **Label Encoding:** Used for ordinal variables like 'class' and 'who', where categories have an inherent order.
- **One-Hot Encoding:** Applied to nominal variables such as 'deck', 'embark\_town', 'sex', and 'embarked', where there is no natural ranking among categories. One-hot encoding created binary columns for each category, making it easier for the model to differentiate between them. Choosing the right encoding technique ensures that categorical data is represented appropriately, preventing biases and improving model performance.

```
Column names in X: Index(['pclass', 'age', 'sibsp', 'parch', 'fare', 'class', 'who', 'adult_male',  
                        'deck', 'embark_town', 'alive', 'alone', 'sex_male', 'embarked_Q',  
                        'embarked_S'],  
                        dtype='object')
```

```
Categorical columns: Index([], dtype='object')
```

Data after encoding:

|   | pclass    | age       | sibsp     | parch     | fare      | class | who | adult_male | \ |
|---|-----------|-----------|-----------|-----------|-----------|-------|-----|------------|---|
| 0 | 0.827377  | -0.592481 | 0.432793  | -0.473674 | -0.502445 | 2     | 1   | True       |   |
| 1 | -1.566107 | 0.638789  | 0.432793  | -0.473674 | 0.786845  | 0     | 2   | False      |   |
| 2 | 0.827377  | -0.284663 | -0.474545 | -0.473674 | -0.488854 | 2     | 2   | False      |   |
| 3 | -1.566107 | 0.407926  | 0.432793  | -0.473674 | 0.420730  | 0     | 2   | False      |   |
| 4 | 0.827377  | 0.407926  | -0.474545 | -0.473674 | -0.486337 | 2     | 1   | True       |   |

|   | embark_town | alive | ... | sex_male | embarked_Q | embarked_S | deck_B | deck_C | \ |
|---|-------------|-------|-----|----------|------------|------------|--------|--------|---|
| 0 | 2           | 0     | ... | True     | False      | True       | False  | False  |   |
| 1 | 0           | 1     | ... | False    | False      | False      | False  | True   |   |
| 2 | 2           | 1     | ... | False    | False      | True       | False  | False  |   |
| 3 | 2           | 1     | ... | False    | False      | True       | False  | True   |   |
| 4 | 2           | 0     | ... | True     | False      | True       | False  | False  |   |

|   | deck_D | deck_E | deck_F | deck_G | deck_Unknown |
|---|--------|--------|--------|--------|--------------|
| 0 | False  | False  | False  | False  | True         |
| 1 | False  | False  | False  | False  | False        |
| 2 | False  | False  | False  | False  | True         |
| 3 | False  | False  | False  | False  | False        |
| 4 | False  | False  | False  | False  | True         |

3. **Addressing Class Imbalance with SMOTE** Class imbalance can negatively impact a machine learning model by making it biased toward the majority class. To overcome this, we used SMOTE (Synthetic Minority Over-sampling Technique), which generates synthetic samples for the minority class to balance the dataset. This method is useful in improving the generalization of the model by ensuring it learns equally from all classes, leading to more accurate predictions.

---

```
Resampled X shape: (1098, 21)
```

```
Resampled y shape: (1098,)
```

```
Classification Report:
```

4. **Model Training and Evaluation** After preprocessing the data and applying SMOTE, we split the dataset into training and testing sets. A Random Forest Classifier was trained using the resampled dataset. To assess the model's effectiveness, we generated a classification report that measured key performance metrics, including precision, recall, and F1-score. This step confirmed

that the feature engineering techniques helped improve the model's ability to classify different categories effectively.

| Classification Report: |           |        |          |         |
|------------------------|-----------|--------|----------|---------|
|                        | precision | recall | f1-score | support |
| 0                      | 1.00      | 1.00   | 1.00     | 103     |
| 1                      | 1.00      | 1.00   | 1.00     | 117     |
| accuracy               |           |        | 1.00     | 220     |
| macro avg              | 1.00      | 1.00   | 1.00     | 220     |
| weighted avg           | 1.00      | 1.00   | 1.00     | 220     |

### Challenges and Solutions:

- Initially, the missing values in the 'deck' column were not properly handled, leading to encoding errors. This was resolved by explicitly adding 'Unknown' as a category before encoding.
- As a solution, several changes were made to improve data preprocessing, handle categorical data, address class imbalance, and build a machine learning model. First, missing values in categorical features like 'deck' were filled with a placeholder value, 'Unknown', to avoid issues with missing data. Ordinal categorical features like 'class' and 'who' were label encoded to transform them into numerical values that preserve their inherent order. Nominal categorical features were one-hot encoded to convert them into binary variables.
- The dataset was imbalanced, which caused biased model predictions. Using SMOTE helped generate a more balanced dataset, improving model performance. To address class imbalance, SMOTE (Synthetic Minority Over-sampling Technique) was used to generate synthetic examples for the minority class, ensuring a more balanced dataset. The data was then split into training and testing sets to evaluate the model's performance.
- Finally, a Random Forest classifier was trained on the resampled data, and its performance was assessed using a classification report, providing key metrics like precision, recall, and F1-score to evaluate the model's effectiveness. These steps ensure that the data is well-prepared for machine learning and that the model can learn effectively from the data.

**Learnings:** This lab highlighted the importance of proper data preprocessing techniques in machine learning. Handling missing values before encoding ensures data consistency, while selecting the right encoding methods helps models interpret categorical data correctly. Addressing class imbalance through techniques like SMOTE improves model fairness and predictive accuracy. Additionally, properly splitting the data into training and testing sets prevents overfitting and ensures reliable performance evaluation.

**Extensions to Other Sampling Techniques:** While SMOTE was effective in this lab, other sampling techniques could be explored for different scenarios:

- Random Undersampling (RUS):** Removes samples from the majority class to achieve balance.

- **ADASYN (Adaptive Synthetic Sampling):** Similar to SMOTE but generates synthetic samples based on the density of minority class instances.
- **Stratified Sampling:** Ensures that the distribution of classes remains proportional in training and testing sets.
- **Hybrid Methods:** Combining oversampling and undersampling techniques can provide better dataset balance and model performance.

**Conclusion:** This lab provided hands-on experience in feature engineering, demonstrating how handling missing data, encoding categorical variables, and balancing class distributions significantly impact model accuracy. By applying techniques such as SMOTE and different encoding strategies, we improved the dataset's quality, leading to better model predictions. This reinforces the importance of data preprocessing as a fundamental step in machine learning. Future improvements could include adding feature scaling and experimenting with alternative sampling methods to further refine model performance.

### ChatGPT prompts used:

#### Initial prompt:

Engineer features to improve model performance using Python.

Handle missing data with Pandas by imputing it with mean or median values.

Scale features using scikit-learn's StandardScaler.

#### 1. Handling Missing Data

- "How can I handle missing values in the 'deck' column of my dataset?"
- "What's the best way to impute missing values in both numerical and categorical columns?"

#### 2. Encoding Categorical Variables

- "How should I encode categorical variables for machine learning?"
- "What's the difference between Label Encoding and One-Hot Encoding, and when should I use them?"
- "How can I one-hot encode nominal categorical variables in a dataset?"

#### 3. Class Imbalance

- "How do I handle class imbalance in my dataset?"
- "Can you show me how to use SMOTE to balance a dataset with class imbalance?"

#### 4. Visualizing Model Performance

- "Can you show me how to plot a confusion matrix for model evaluation?"
- "How can I visualize feature importance for a Random Forest model?"
- "How can I plot ROC curves and calculate the AUC score for my model?"

#### **5. Alternative Sampling Techniques**

- "What are other sampling techniques I can use to address class imbalance besides SMOTE?"
- "Can you explain how Random Undersampling works and when to use it?"
- "What is ADASYN, and how does it differ from SMOTE?"
- "How can I implement Stratified Sampling to maintain class distribution in training and testing sets?"

**References:** ChatGPT